

Automation Workflow Templates

12 ready-to-build workflows for Make.com & Zapier | By Samme Samuel, CoreFlare Agency

Before You Build

These are specs, not screenshots. Every template below lists the trigger, the apps, and the modules in order. Open Make.com or Zapier, follow the steps, swap in your own field names. That's it.

How to adapt them. Three things change per business:

- **Field names.** My *Full Name* is your *contact_name*. Map to what your form and CRM actually call things. Never assume the field label matches the API key — check the raw output of your trigger module first.
- **The router branches.** Most of these split on a condition (lead score, service type, form source). Your split criteria will differ. The structure won't.
- **The app.** GoHighLevel here can be HubSpot there. Airtable can be Google Sheets. The logic survives the swap. The field mapping doesn't — redo it.

Build in Make.com or Zapier? Quick rule: if the workflow has branching, loops, or more than about eight steps, build it in Make.com. Visual routers and iterators make complex logic readable. If it's a straight line from A to B and non-technical people need to maintain it, Zapier is faster to ship. Both are listed for every template.

Error handling: do this on every single one

Most broken automations aren't broken logic. They're unhandled failures that nobody noticed for three weeks. Build these habits into every template in this document:

- **Add an error handler to every module that touches an external API.** In Make.com, right-click the module and add an error handler route. Use *Break* with auto-retry for anything transient (rate limits, timeouts) and *Resume* with a fallback value for anything optional (enrichment data, AI output).
- **Route failures somewhere a human looks.** A Slack channel or an "Automation Errors" sheet. Log the scenario name, the timestamp, the record ID, and the error text. An error you can't trace back to a record is useless.
- **Never let one failed record kill the run.** When processing a list, wrap the per-record logic so a single bad row doesn't halt the iteration.

- **Turn on incomplete executions in Make.com.** Failed runs get stored instead of vanishing. You can fix the data and re-run them.
- **Deduplicate before you write.** Every workflow that creates a CRM record should search first, then branch: found = update, not found = create. This one habit prevents most of the mess in this document.
- **Test with real data before you turn it on.** Run it once manually. Check the record it created. Then activate.

Now build.

1. Website Form to CRM Sync

What it does: Pushes every website form submission into your CRM as a properly tagged contact, with no duplicates.

Who it's for: Anyone still exporting form entries to CSV and importing them by hand.

Trigger	New form submission received (webhook from your website form, or Watch Responses in Google Forms)
Apps used	Make.com or Zapier, website form / Google Forms, GoHighLevel, Google Sheets
What it saves	Removes manual copy-paste between form and CRM, and eliminates the duplicate contacts created when the same person submits twice

Step-by-step build

1. **Trigger:** Custom Webhook (Make.com) or Catch Hook (Zapier). Point your form's webhook URL at it. Submit one real test entry so the platform learns the data structure.
2. **Tools > Set Variables:** normalize the incoming data. Lowercase and trim the email (`lower(trim(email))`), strip spaces and dashes from the phone, and title-case the name. Dirty input is the number one cause of duplicate contacts.
3. **GoHighLevel > Search Contacts:** query by the normalized email.
4. **Router** with two branches based on the search result.
5. **Branch A (contact found):** GoHighLevel > Update Contact. Map only the fields that add information — new phone, new form message. Do not blank out existing fields with empty form values. Add tag `repeat-inquiry`.
6. **Branch B (no contact found):** GoHighLevel > Create Contact. Map: First Name, Last Name, Email, Phone, Source = form name, plus a custom field for the message body. Add tags `web-form` and the specific form name.

7. **Both branches:** Google Sheets > Add Row to a "Lead Log" sheet. Columns: timestamp, name, email, phone, source, CRM contact ID, new-or-existing. This is your audit trail when someone says "the lead never came through."
8. **Error handler** on both GoHighLevel modules: route to a Slack message or an error sheet with the raw webhook payload attached, so nothing is lost.

Watch out for: Duplicate contacts from inconsistent email casing. *Sam@Company.com* and *sam@company.com* are different strings to a search module, so step 2 is not optional. Second issue: some form tools fire the webhook twice on a slow connection. Add a check against your Lead Log for the same email within the last 60 seconds and stop the run if it matches.

2. Speed-to-Lead Instant Alert

What it does: The second a new lead comes in, it texts and emails them a reply and pings you on your phone.

Who it's for: Service businesses where the first responder wins the deal.

Trigger	New contact created in GoHighLevel with tag new-lead
Apps used	Make.com or Zapier, GoHighLevel (SMS + email), Slack, Google Sheets
What it saves	Removes the delay between a lead arriving and a human noticing it — no more leads sitting unread in an inbox overnight

Step-by-step build

1. **Trigger:** GoHighLevel > Watch Contacts, filtered to the new-lead tag. Set the polling interval to the shortest your plan allows, or use a GHL workflow webhook for true instant firing.
2. **Filter:** continue only if Phone is not empty AND the contact does not already carry the tag speed-to-lead-sent. This is your re-trigger guard.
3. **Tools > Set Variable:** compute local hour for the lead using the business timezone, not the scenario timezone. In Make.com: `formatDate(now; "H"; "Africa/Douala")` — replace with your operating timezone.
4. **Router** with two branches on that hour value.
5. **Branch A (inside business hours):** GoHighLevel > Send SMS. Short and human: name, what they enquired about, a question back. Then GoHighLevel > Send Email with the fuller version and a booking link.
6. **Branch B (outside business hours):** GoHighLevel > Send Email only. Then Tools > Sleep or a scheduled follow-up so the SMS fires at opening time. Nobody wants a sales text at 2am.

7. **Slack > Create Message** to your #leads channel: name, phone, service requested, source, and a direct link to the CRM record.
8. **GoHighLevel > Add Tag** speed-to-lead-sent to the contact. This closes the loop on step 2.
9. **Google Sheets > Add Row:** log the lead timestamp and the response timestamp so you can actually measure your response lag.

Watch out for: Timezone drift. Make.com scenarios run in the timezone set on your organisation, which is often UTC, not yours. Hardcode your business timezone into every date calculation. Also watch double-firing: if a GHL workflow updates the contact right after creation, a "watch contacts" trigger can fire twice. The tag guard in steps 2 and 8 is what stops the same lead getting texted twice.

3. Calendar Booking to CRM and Reminder Sequence

What it does: Turns a Calendly booking into a CRM opportunity plus a three-touch reminder sequence that cuts no-shows.

Who it's for: Consultants, coaches, and agencies who run discovery calls.

Trigger	Calendly: Invitee Created (new booking)
Apps used	Make.com or Zapier, Calendly, GoHighLevel, Google Calendar, Slack
What it saves	Removes manual CRM entry after every booking and eliminates the manual reminder emails you send the day before each call

Step-by-step build

1. **Trigger:** Calendly > Watch Events (Invitee Created). Pull the full payload including questions and answers — that's where your qualification data lives.
2. **GoHighLevel > Search Contacts** by invitee email. Router: found = update, not found = create. Same dedupe pattern as Template 1.
3. **GoHighLevel > Create Opportunity** in your sales pipeline, stage "Call Booked". Map: contact ID, opportunity name = {{name}} - Discovery Call, and the call datetime into a custom field.
4. **Tools > Set Variables:** build the reminder timestamps from the booking start time. reminder_1 = start - 24 hours, reminder_2 = start - 1 hour. Store the invitee's own timezone from the Calendly payload — you will need it in step 6.
5. **Google Calendar > Create Event** (if Calendly isn't already syncing). Include the invitee's answers in the description so you walk in prepared.

6. **GoHighLevel > Add Contact to Workflow** — a GHL workflow holding your three touches:
 - (a) instant confirmation email with the agenda and reschedule link, (b) 24-hour reminder email, (c) 1-hour reminder SMS. Render all times in the invitee's timezone, not yours.
7. **Slack > Create Message** to #bookings: who, when, what they answered.
8. **Second scenario — cancellations:** Calendly > Invitee Canceled. Move the opportunity to "Cancelled", remove the contact from the reminder workflow, notify Slack. Skip this and you will send reminders for calls that no longer exist.

Watch out for: Timezone rendering. Calendly returns times in UTC. If you drop that raw value into a reminder email, someone in another country gets the wrong hour and misses the call. Always format using the `invitee.timezone` field from the payload. Second trap: reschedules. Calendly sends a cancel plus a new booking, so without the cancellation scenario you end up with two opportunities and two reminder sequences for one person.

4. Form to Proposal and Invoice

What it does: Takes a completed scoping form and generates a personalised proposal document and a draft invoice, ready for your review.

Who it's for: Freelancers and agencies rebuilding the same proposal from scratch every week.

Trigger	New row in the Airtable "Scoping Requests" table with Status = <i>Ready for Proposal</i>
Apps used	Make.com or Zapier, Airtable, Google Docs, Google Drive, OpenAI, Gmail, GoHighLevel
What it saves	Removes rebuilding the same proposal document by hand and re-typing scope details into an invoice

Step-by-step build

1. **Trigger:** Airtable > Watch Records on a view filtered to Status = *Ready for Proposal*. Using a filtered view instead of a formula filter keeps the scenario clean and cheap.
2. **Router** on the Service Type field — one branch per offer (automation build, website, GHL setup). Each branch points at a different Google Docs template. Don't try to make one template cover everything.
3. **OpenAI > Create a Chat Completion:** generate only the bespoke paragraphs — the problem summary and the recommended approach. Feed it the client's own words from the form. Keep pricing, terms, and scope tables out of the AI's hands entirely; those are fixed template blocks.
4. **Google Docs > Create a Document from Template.** Map placeholders: `{{client_name}}`, `{{company}}`, `{{problem_summary}}` (from step 3), `{{approach}}` (from step 3), `{{deliverables}}`, `{{timeline}}`, `{{investment}}`, `{{valid_until}}`. Name the file Proposal – `{{company}}` – `{{date}}`.

5. **Google Drive > Move File** into a client folder, and set sharing to "anyone with the link can view".
6. **Airtable > Update Record:** write the doc URL back, set Status = *Proposal Drafted*, stamp the date.
7. **Create the draft invoice** — an Airtable record in an "Invoices" table (or a GoHighLevel invoice), with line items from the scoping form, status Draft. Draft, never sent automatically.
8. **Gmail > Create Draft** addressed to you, not the client, containing the proposal link. You read it, you send it. AI-generated text goes out under your name only after your eyes are on it.

Watch out for: Auto-sending. The single biggest failure mode here is a proposal with a hallucinated deliverable or a wrong number reaching a client. Keep step 8 as a draft, permanently. Also: OpenAI calls can time out or return oversized text — set a token cap, add an error handler that falls back to a generic paragraph rather than leaving the placeholder unfilled, and check the doc for stray `{{ }}` markers before sending.

5. Social Content Scheduler from a Content Calendar

What it does: Reads your content calendar every morning and pushes the day's posts to each platform, reformatted per channel.

Who it's for: Solo operators and small teams posting across several platforms.

Trigger	Scheduled — daily at a fixed hour
Apps used	Make.com or Zapier, Airtable or Notion, OpenAI, Slack, your social platform connectors
What it saves	Removes logging into each platform separately and manually reformatting the same post for different character limits

Step-by-step build

1. **Trigger:** Schedule module, once daily, early. Set the scenario timezone explicitly.
2. **Airtable > Search Records:** filter to Publish Date = TODAY() AND Status = *Approved*. Approved-only is deliberate — drafts should never be publishable by accident.
3. **Iterator:** loop over the returned records so each post is handled independently.
4. **Router** per platform, driven by a multi-select Channels field on the record.
5. **OpenAI > Create a Chat Completion** per branch: reformat the master copy for that channel's constraints. Give it the actual limits in the prompt — LinkedIn long-form with line breaks, short punchy for X, hook-first caption plus hashtags for TikTok. Cap output length in the prompt itself, don't rely on truncation.

6. **Platform publish module** per branch. Attach the media file from the Airtable attachment field.
7. **Airtable > Update Record:** Status = *Published*, write the timestamp and the returned post URL per channel.
8. **Aggregator > Slack > Create Message:** one daily digest listing what went out and any failures. Not one Slack ping per post.
9. **Error handler** on each publish module: set that record's Status to *Publish Failed*, log the error text into the record, and let the iterator continue to the next post.

Watch out for: Platform rate limits and API rejections — oversized images, unsupported video codecs, too many hashtags. Without step 9, one bad record stops the whole run and the rest of the day's posts silently never publish. Also watch the date filter: if the scenario timezone and your Airtable timezone disagree, a scheduled post fires a day early or gets skipped entirely.

6. Inbound Email Parsing to Task Creation

What it does: Reads incoming client emails, extracts the actual request, and creates a properly assigned task with a due date.

Who it's for: Anyone whose to-do list is really just their inbox.

Trigger	New email in Gmail matching a label or search query
Apps used	Make.com or Zapier, Gmail, OpenAI, Notion or Airtable, Slack
What it saves	Removes re-typing email requests into a task manager and stops requests getting buried under newer mail

Step-by-step build

1. **Trigger:** Gmail > Watch Emails, scoped by a search query such as `label:client-requests is:unread`. Mark as read on processing so you don't re-handle the same message.
2. **Filter:** skip anything where the sender matches your automated-sender list (no-reply addresses, notifications, newsletters). Otherwise you'll be creating tasks from Stripe receipts.
3. **Text parser > Replace:** strip the quoted reply chain. Cut everything from the first `On [date] ... wrote:` marker onward. Sending a 40-message thread to the AI is expensive and gives worse results.
4. **OpenAI > Create a Chat Completion** with JSON mode on. Ask for a strict object: `{ "is_actionable": bool, "task_title": string, "summary": string, "urgency": "low|medium|high", "due_date": "YYYY-MM-DD or null", "client": string }`. Instruct it to return `is_actionable: false` for FYI-only mail.

5. **JSON > Parse JSON** on the response. Do not skip this and try to regex the raw string.
6. **Filter:** continue only if `is_actionable` is true.
7. **Notion > Create a Database Item** (or **Airtable > Create Record**). Map: Title, Summary, Urgency, Due Date, Client, Source Email link
(`https://mail.google.com/mail/u/0/#inbox/{{threadId}}`), Status = *To Do*.
8. **Router on urgency:** if *high*, Slack > Create Message with a direct link to the task and the email. Medium and low just land in the board.
9. **Gmail > Add Label** `task-created` so you can see at a glance what was captured.

Watch out for: The model returning prose instead of JSON, which breaks the parse module. Use JSON mode, and add an error handler on step 5 that falls back to creating a task with the raw subject line as the title — capturing it imperfectly beats losing it. Also watch trigger loops: if your task tool emails a confirmation to the same inbox, you'll create a task about a task. Exclude that sender in step 2.

7. Abandoned Form Follow-Up

What it does: Catches people who started your form but never finished, and sends them a nudge.

Who it's for: Anyone running a multi-step form, quiz, or application.

Trigger	Webhook fired on partial form completion (email captured, form not submitted)
Apps used	Make.com or Zapier, multi-step form tool, Airtable, GoHighLevel, Mailgun
What it saves	Removes the need to manually spot and chase half-finished submissions — most of which are simply never seen at all

Step-by-step build

1. **Form setup first:** put the email field on step one and enable partial-submission webhooks. If your form tool can't emit partial data, this template can't work — switch tools or add a two-stage form where step one submits on its own.
2. **Trigger:** Custom Webhook receiving the partial payload.
3. **Airtable > Upsert Record** in a "Form Sessions" table, keyed on session ID. Fields: email, name, last step reached, first-seen timestamp, updated timestamp, Status = *Partial*. Upsert, not create — a person moving from step 2 to step 3 must update one row, not make two.
4. **Second webhook scenario, on full submission:** Airtable > Update Record for that session ID, Status = *Completed*. This is what protects you from chasing someone who already finished.
5. **Third scenario — the chase.** Schedule module, hourly.

6. **Airtable > Search Records:** Status = *Partial* AND updated timestamp older than 2 hours AND Nudge Sent is empty. The delay matters — someone might just be slow, or on a bad connection.
7. **Iterator** over the results.
8. **Mailgun > Send Email** (or GoHighLevel email). Reference where they stopped: "You got as far as the budget question." Include a resume link with the session ID appended so they don't restart from zero.
9. **Airtable > Update Record:** stamp Nudge Sent. Without this, the hourly scenario emails the same person every hour. This is the mistake that gets domains blocked.
10. **Optional second nudge** after 48 hours with different copy, then set Status = *Abandoned* and stop. Two touches, then leave them alone.

Watch out for: Emailing people who already converted. If scenario 4 fails or lags, the chase scenario sees a stale *Partial* status and nudges a paying customer about an unfinished form. Add a defensive CRM lookup in step 8 — if the contact already has a customer tag, skip. And guard step 9 hard: a missing Nudge Sent stamp turns a helpful reminder into an hourly spam loop.

8. Post-Service Review Request

What it does: Asks for a review at the right moment, and routes unhappy responses to you instead of to Google.

Who it's for: Local services, agencies, anyone whose reputation is public.

Trigger	Opportunity moved to stage <i>Completed</i> in GoHighLevel
Apps used	Make.com or Zapier, GoHighLevel, Google Sheets, Slack, Mailgun
What it saves	Removes remembering to ask each client individually, and stops negative feedback going straight to a public review page

Step-by-step build

1. **Trigger:** GoHighLevel > Watch Opportunities, filtered to stage *Completed*.
2. **Filter:** skip if the contact carries tag *review-requested* in the last 90 days. Asking the same repeat client every single job is how you get ignored.
3. **Tools > Sleep** or a scheduled delay of 24 to 48 hours. Immediately after delivery is too soon; a week later they've moved on.
4. **GoHighLevel > Send SMS:** one question, nothing else. "Quick one — how did we do, 1 to 10?" No links yet. You're gating on the answer.

5. **Second scenario:** GoHighLevel > Watch Inbound Messages from contacts tagged awaiting-review-reply.
6. **Text parser:** extract the first number from the reply body.
7. **Router** on that number.
8. **Branch A (9 or 10):** send the public review link — Google Business, Trustpilot, wherever it counts. Tag promoter.
9. **Branch B (1 to 8):** no public link. Send a personal reply asking what fell short. Slack > Create Message to #service-recovery with the contact, the score, and the job. Tag needs-followup. A human calls them.
10. **Branch C (no number found):** route to Slack for a human to read. Don't let a parser guess at a real customer's words.
11. **Google Sheets > Add Row:** log every score with date, client, and job type. That's your satisfaction trend.
12. **Add tag** review-requested with the date, closing the loop on step 2.

Watch out for: Free-text replies that break the parser — "8 or 9, honestly", "great!", or an emoji. Always build branch C. A misrouted 4 that receives a public review link is a genuinely expensive bug. Also check your local rules on review gating: some platforms prohibit selectively soliciting only positive reviewers, so confirm the policy for the specific platform you're pointing at.

9. Lead Enrichment on Capture

What it does: Fills in company details on every new lead automatically, so you know who you're talking to before you reply.

Who it's for: B2B teams who currently Google every inbound lead by hand.

Trigger	New contact created in GoHighLevel from a B2B form source
Apps used	Make.com or Zapier, GoHighLevel, HTTP module, an enrichment provider, OpenAI, Airtable, Slack
What it saves	Removes manually researching each lead's company, size, and industry before the first call

Step-by-step build

1. **Trigger:** GoHighLevel > Watch Contacts, filtered to your B2B form sources.
2. **Tools > Set Variable:** extract the domain from the email — everything after the @.
3. **Filter:** stop if the domain is in a free-provider list (gmail.com, yahoo.com, outlook.com, hotmail.com, icloud.com). Enriching a Gmail address burns credits and returns nothing. Tag those contacts `personal-email` and let them through unenriched.

4. **HTTP > Make a Request** to your enrichment provider with the domain. Set a timeout and turn off "stop on error" — enrichment is a nice-to-have, not a blocker.
5. **Error handler on step 4:** Resume with an empty structure. The lead must reach your CRM whether or not enrichment succeeds. Never let a third-party API failure lose you a lead.
6. **OpenAI > Create a Chat Completion** (optional): pass the company description and return a one-line "why they might need us" plus an industry category from your own fixed list. Constrain the categories in the prompt — free-form output is useless for filtering later.
7. **GoHighLevel > Update Contact:** write company name, employee count band, industry, website, and location into custom fields. Only overwrite fields that are currently empty — never clobber something a human typed.
8. **Tools > Set Variable — lead score:** a simple additive rule. Employee count in your target band +30, target industry +30, business domain +20, requested a demo +20. Keep it readable; you'll tune it monthly.
9. **Airtable > Create Record** in a "Enriched Leads" table with the full profile and score, for pipeline reporting.
10. **Filter:** if score is at or above your hot threshold, pass to Template 10 (hot lead alerting) via a webhook.

Watch out for: Credit burn and rate limits. Enrichment APIs are metered and most rate-limit aggressively. Step 3 is what keeps the bill sane. Also cache: store enriched domains in Airtable and check there before calling the API — five leads from the same company should cost you one lookup, not five. And treat missing enrichment as normal, not as an error; small and new companies simply aren't in these databases.

10. Hot Lead Slack Alert

What it does: Pushes only high-intent leads into Slack with full context and a one-click claim button.

Who it's for: Any team where leads get missed because everyone assumed someone else had it.

Trigger	Contact score crosses the hot threshold, or a high-intent action fires (pricing page visit, demo request, reply to outreach)
Apps used	Make.com or Zapier, GoHighLevel, Instantly, Slack, Google Sheets
What it saves	Removes the need to watch a CRM dashboard for buying signals, and ends the "I thought you were handling that one" problem

Step-by-step build

1. **Trigger:** multiple entry points into one webhook — Template 9's scoring output, an Instantly "reply received" webhook, a GHL tag-added event for requested-demo. One alerting scenario, many sources.
2. **Tools > Set Variable:** normalise the payloads into a common shape — name, email, company, signal type, score, CRM link. Do this once at the top and the rest of the scenario stays simple.
3. **Filter:** continue only if score is above threshold OR the signal is inherently high-intent (demo request, positive reply).
4. **Data store > Get Record** (Make.com) keyed on email: if this contact was alerted in the last 24 hours, exit. Alert fatigue is real — the moment your team starts ignoring the channel, the automation is worthless.
5. **GoHighLevel > Get Contact** for the full record: source, tags, last activity, enrichment fields.
6. **Slack > Create Message** to #hot-leads using Block Kit. Include: name and company as the header, the signal that fired, the score, the enriched context, the last thing they did, and buttons linking to the CRM record and the Calendly booking link. All the context in the message — no one should have to open three tabs.
7. **Slack > Add Reaction** — a placeholder emoji establishing the claim convention. Whoever reacts owns it. Simple, and it works without building a Slack app.
8. **Data store > Add Record:** stamp email plus timestamp for the step 4 guard.
9. **Google Sheets > Add Row:** log every alert with signal type and time, so you can check later whether hot leads are actually getting worked.
10. **Optional escalation scenario:** run every 30 minutes, find alerts with no reaction after an hour, and re-post as a direct mention to the on-duty rep.

Watch out for: Over-alerting. If every form fill hits #hot-leads, the channel gets muted within a week and you're worse off than before. Set the threshold high, review it monthly, and keep the dedupe guard in step 4. Also watch Slack rate limits on bursts — if a campaign send triggers 50 replies at once, add an aggregator so they arrive as one summarised message rather than 50 pings.

11. Weekly KPI Digest to Email and Sheets

What it does: Pulls your numbers from every system every Monday, writes them to a tracking sheet, and emails you a plain-language summary.

Who it's for: Operators who currently rebuild the same report by hand every week — or, more honestly, skip it.

Trigger	Scheduled — weekly, Monday morning
Apps used	Make.com or Zapier, GoHighLevel, Instantly, Google Sheets, OpenAI, Gmail or Mailgun, Slack

What it saves	Removes exporting from four dashboards and re-keying the numbers into a spreadsheet every week
----------------------	--

Step-by-step build

1. **Trigger:** Schedule module, Monday at a set hour, timezone locked to your business.
2. **Tools > Set Variables:** compute the reporting window. `week_start = now - 7 days at 00:00`, `week_end = now at 00:00`. Use a fixed boundary, not a rolling one, or your week-over-week comparisons are meaningless.
3. **GoHighLevel > Search Contacts** created in that window. Follow with an Aggregator to get a count, and group by source.
4. **GoHighLevel > Search Opportunities** in the window: count created, count won, and total value by pipeline stage.
5. **Instantly > Get Campaign Analytics** for the window: sent, opened, replied, per campaign.
6. **Google Sheets > Get Range Values** from your KPI history tab to pull last week's row for comparison.
7. **Tools > Set Variables:** calculate deltas. Guard against divide-by-zero — if last week's value is 0, output "n/a" rather than letting the module error out.
8. **Google Sheets > Add Row** to KPI History. One row per week, one column per metric. This sheet is the asset; the email is just the reminder to look at it.
9. **OpenAI > Create a Chat Completion:** pass the numbers and the deltas, ask for three sentences — what moved, what didn't, what deserves attention. Explicitly instruct it to use only the numbers provided and never estimate or extrapolate.
10. **Gmail or Mailgun > Send Email** to yourself: a simple HTML table of metrics with deltas, the AI summary underneath, and a link to the sheet.
11. **Slack > Create Message** with the three headline numbers for the team.
12. **Error handler on every data-source module:** Resume with a null and mark that metric as "unavailable" in the output. A partial report on time beats no report at all.

Watch out for: Silent zeros. If an API call fails and you write 0 to the sheet, your historical trend is permanently corrupted and you'll make decisions on it. Always distinguish "genuinely zero" from "we couldn't fetch it" — step 12 is why. Second issue: pagination. Search modules return a capped page by default, so a busy week gets undercounted. Set the record limit high or loop until the cursor is exhausted. Third: timezone boundaries shift your week edges — lock the timezone in step 1 and never change it, or the history stops being comparable.

12. Client Onboarding Sequence

What it does: Turns a signed deal into a fully provisioned client — folders, project board, kickoff booking, welcome sequence — automatically.

Who it's for: Agencies and service providers repeating the same 15-step setup for every new client.

Trigger	Opportunity moved to stage <i>Won</i> in GoHighLevel
Apps used	Make.com or Zapier, GoHighLevel, Google Drive, Notion or Airtable, Slack, Gmail, Calendly
What it saves	Removes the manual checklist of creating folders, boards, and accounts for each new client — and stops steps getting skipped when you're busy

Step-by-step build

1. **Trigger:** GoHighLevel > Watch Opportunities, stage = *Won*.
2. **Filter:** stop if a client record with this contact ID already exists in your Clients table. Re-dragging a card in the pipeline must not provision the same client twice.
3. **Tools > Set Variables:** build a clean client slug from the company name — lowercase, hyphens, no special characters. Use it consistently for folder names, board names, and channel names. Consistency here is what makes everything findable in a year.
4. **Google Drive > Create Folder** under Clients, named with the slug. Then create the subfolders you always need: Contracts, Assets, Deliverables, Credentials.
5. **Google Drive > Copy File** for your standard onboarding docs into the new folder — intake questionnaire, scope summary, access checklist.
6. **Notion > Create a Page from a template** (or Airtable > Create Records from a task template). Map client name, start date, package, and owner. Generate the standard task list with due dates offset from the start date.
7. **Airtable > Create Record** in your master Clients table: name, contact, package, start date, Drive folder URL, Notion page URL, status = *Onboarding*. This record is the hub — everything else links back to it.
8. **Slack > Create Channel** named `client-{{slug}}`, then post a pinned message with all the links from steps 4, 6, and 7.
9. **GoHighLevel > Add Contact to Workflow** — your welcome sequence: (a) instant welcome email with the kickoff booking link and the intake form, (b) day-2 nudge if the intake form is still unsubmitted, (c) day-5 what-happens-next email.
10. **Gmail > Create Draft** of a personal welcome note from you. One human message in an otherwise automated sequence is what stops onboarding feeling like a machine.
11. **GoHighLevel > Add Tag** `client-onboarded`, and update the contact record with the client Airtable ID.
12. **Companion scenario — the stall check.** Runs daily: find clients in *Onboarding* status for more than 7 days with no kickoff call booked, and post them to Slack for a human to chase. Automation starts onboarding; a person still has to finish it.

Watch out for: Double provisioning. Opportunity stages get dragged, reverted, and re-dragged. Without the guard in step 2 you end up with two Drive folders, two Slack channels, and two welcome emails to a brand-new client. Also watch API failure part-way through: if Notion fails at step 6, you have a Drive folder and no board. Add error handlers that log exactly which step failed to a Slack channel with the client slug, so the gap can be filled manually. And check permissions before you go live — Drive folder creation and Slack channel creation both need scopes that a default connection often lacks.

Build One This Week

Don't build all twelve. Pick the one that maps to the task you're most tired of doing, and ship it this week. A single working automation beats a folder of half-built scenarios.

My order of operations for most businesses: **Template 1** first, because clean data in the CRM makes everything downstream possible. Then **Template 2**, because speed to lead is the cheapest revenue lever you have. Then whichever of the remaining ten hurts most.

And map the customer journey before you build anything. Automation applied to a broken process just breaks it faster.

Want these built and running in your stack?

I build automation systems for entrepreneurs and agencies — GoHighLevel, Make.com, Zapier, cold email infrastructure, CRM setup, and AI-powered workflows. If you'd rather have these implemented, tested, and handed over working, let's talk.

- **Email:** selangwe19u@gmail.com
- **Book a call:** <https://calendly.com/selangwe19u/30min>
- **Website:** <https://samme-samuel.coreflareagency.com/>
- **LinkedIn:** <https://www.linkedin.com/in/samme-samuel-975a8b305>

Automation Workflow Templates — by Samme Samuel, CoreFlare Agency. AI Automation, GoHighLevel, and lead generation systems for businesses that want to stop doing the same task twice.